Documento de Arquitectura de Software (DAS / SAD)

arc42 simplificado + Modelo C4 + ADRs + Corte Vertical

SITRA

Servicio de Integración y Trazabilidad

Exposición segura y auditable de un núcleo transaccional legado a una billetera digital externa

Caso base: Caso 5 — Cooperativa de ahorro y crédito

Curso: Arquitectura de Software — Proyecto individual
Estudiante: Jorge Mario Vélez Vásquez
Proyecto: SITRA — Servicio de Integración y Trazabilidad
Repositorio: https://github.com/JorgeMario-V/sitra
Versión del documento: 1.0 — Fecha: 07 / 11 / 2026

Historial de versiones

Versión	Fecha	Contenido	Clase
0.1	26/09/2026	Ficha de drivers revisada + Sección 1 (visión, historias de usuario, atributos de calidad, restricciones).	Clase 1
0.4	17/10/2026	Sección 2 (patrón y pila); 3.1 y 3.2 (contexto y contenedores); 3.3 y 3.4 (clases y secuencia); 5.2 (script SQL DDL). Ajuste en 1.3: nombre y fórmula de la métrica principal.	Clases 2, 3 y 4
0.5	24/10/2026	Sección 4 (ADR-001 a 003), 3.5 (despliegue), arranque del corte vertical y cambio a MariaDB/XAMPP (ADR-002). El modelo incorpora Asociado, intentos RECHAZADOS e IP de origen.	Clase 5
0.6	07/11/2026	CRUD punta a punta de Transferencia (5.3) con token JWT + BCrypt (ADR-004 y ADR-005), consulta de saldo (HU-02) y consulta por fechas + verificación de integridad de la bitácora (HU-03).	Clase 6
1.0	07/11/2026	Pruebas de la arquitectura (5.4: JUnit + Postman), análisis de trade-offs, puntos de sensibilidad y conclusión (Sección 6), lista de chequeo y README de instalación. Documento completo, Secciones 1 a 6.	Clases 7 y 8 (adelantadas)

SECCIÓN 1: INTRODUCCIÓN Y DRIVERS ARQUITECTÓNICOS

1.1 Visión general del negocio y alcance

Problema de negocio

Una cooperativa de ahorro y crédito opera sobre un núcleo transaccional construido hace 20 años en un lenguaje propietario, cuya documentación se perdió y que nadie del equipo actual escribió. El regulador le exige dos cosas en un plazo fijo de 6 meses: integrarse con una billetera digital externa y entregar trazabilidad de sus transacciones. Si no cumple, arriesga la licencia de operación.

Hoy el problema lo padecen el área de cumplimiento y la gerencia. Sin software, la evidencia para el auditor se reconstruye a mano con reportes parciales del núcleo y conciliaciones en hojas de cálculo: un proceso lento, que no demuestra que los datos no fueron alterados y que no alcanza los tiempos del auditor. La exposición de funciones a un tercero, simplemente, no existe.

Además, el cliente fijó tres condiciones que no se discuten: el núcleo no se va a cambiar (lo seguirán usando porque su negocio no es el software), no hay dinero adicional ni se invertirá en APIs (solo se trabaja con lo que se tiene) y el personal de la cooperativa se dedica a sus funciones, no a desarrollar ni mantener software.

Propuesta de solución

El sistema permite a la billetera digital externa, autenticada, consultar saldos y registrar transferencias sobre cuentas autorizadas de la cooperativa, para entregar al auditor el 100 % de esas operaciones con evidencia verificable de que no fueron alteradas, sin modificar el núcleo legado y sin inversión adicional.

Alcance (in-scope)

- Conector externo al núcleo que usa solo lo que el núcleo ya tiene hoy (sus salidas e interfaces actuales); no se compra ni se instala ningún producto.
- Exposición de dos operaciones a la billetera: consulta de saldo y registro de transferencias (con su corrección, reintento y anulación).
- Autenticación de la billetera con token JWT y autorización por rol (BILLETERA y AUDITOR).
- Lista blanca de cuentas autorizadas: solo esas cuentas son visibles para el tercero.
- Bitácora de auditoría que no se puede modificar ni borrar, consultable y verificable por el auditor.
- Simulador del núcleo para poder probar sin tocar el sistema en producción.

Fuera de alcance (out-of-scope)

- Modificar, reescribir o reemplazar el núcleo legado. Tampoco se plantea una migración gradual: el núcleo se queda indefinidamente.
- Cualquier otra función del núcleo distinta de saldo y transferencias.
- Aplicación móvil o interfaz para el usuario final: el consumidor es la billetera (sistema a sistema).
- Productos o licencias de pago: API Gateway comercial, servicios de nube facturados, herramientas de auditoría de pago.
- Alta disponibilidad (réplicas, balanceo) y reportería avanzada o tableros analíticos.
- Contratar soporte externo: no es una decisión de arquitectura; se eleva a la junta directiva (ver 1.4).

1.2 Historias de usuario arquitectónicamente significativas

ID	Historia de usuario	¿Por qué es arquitectónicamente significativa?
HU-01	Como billetera digital externa autenticada, quiero registrar una transferencia sobre una cuenta autorizada, para que se ejecute en el núcleo y quede	Es la más riesgosa: recorre todas las capas y ejercita a la vez autenticación, validación, integración con el núcleo legado, persistencia y bitácora. Es el Corte

ID	Historia de usuario	¿Por qué es arquitectónicamente significativa?
	con evidencia auditable.	Vertical (Sección 5).
HU-02	Como billetera digital externa autenticada, quiero consultar el saldo de una cuenta autorizada, para mostrárselo al usuario final.	Ejercita la integración de solo lectura con el núcleo y el mínimo privilegio (lista blanca). Valida que el saldo no se copia: siempre se lee del núcleo, que es la única fuente de verdad.
HU-03	Como auditor, quiero consultar las operaciones por cuenta y rango de fechas y verificar su integridad, para entregar la evidencia que exige el regulador.	Materializa el atributo crítico del caso (trazabilidad). Ejercita el control por rol y el mecanismo que demuestra que un registro no fue alterado.

Criterio: las tres validan el patrón, la seguridad, la persistencia y la integración con un sistema externo. Se descartaron historias como "ver el estado del servicio" o "listar cuentas" por no ser significativas.

1.3 Atributos de calidad prioritarios (ISO/IEC 25010) — especificación cuantitativa

La prioridad la da el caso: seguridad y trazabilidad "por encima de cualquier otro atributo". La fiabilidad sigue porque se mueve dinero de asociados, y la mantenibilidad sube a primer orden porque no habrá personal técnico que sostenga el sistema.

#	Atributo (ISO 25010)	Escenario de calidad (estímulo → entorno → respuesta)	Métrica objetivo (medible)	¿Cómo se verificará?
AC-01	Seguridad — responsabilidad, no repudio, integridad (ficha: AC-06)	La billetera hace operaciones en operación normal; además, un cliente sin token intenta registrar una transferencia y alguien intenta modificar un registro de la bitácora directamente en la base de datos.	Métrica principal — Completitud del registro de auditoría (ISO/IEC 25023): X = A / B, donde A = operaciones registradas en la bitácora y B = operaciones realizadas. Meta: X = 1 (100 %). Además: 0 registros de la bitácora modificables o borrables; 401 en el 100 % de los intentos sin token; la consulta del auditor por cuenta y fechas responde en ≤ 5 s.	Colección Postman (peticiones 03, 17, 18 y 19); prueba de integración JUnit; db/pruebas/demo_sustentacion.sql (UPDATE y DELETE sobre la bitácora rechazados; verificación de huellas); tiempo de respuesta medido en Postman.
AC-02	Fiabilidad — tolerancia a fallos (ficha: AC-05)	El núcleo legado no responde mientras se procesa una transferencia ya aceptada. El sistema la cierra sin dejarla a medias.	100 % de estas transferencias terminan en estado FALLIDA con respuesta 503, sin efecto parcial. Repetir la misma referencia responde 409 y nunca duplica la operación. Espera máxima al núcleo: 5 s.	Prueba unitaria con el núcleo sustituido (TransferenciaServiceTest) + modo "sin-respuesta" del simulador + Postman (petición 05, 409).
AC-03	Mantenibilidad — modularidad y capacidad de ser probado (ficha: AC-07)	Se necesita probar sin el núcleo real (no hay ambiente de pruebas) o cambiar la forma de conectarse a él, y no hay personal técnico que conozca el núcleo.	El cambio toca como máximo 1 clase del conector y 1 valor de configuración, y 0 archivos de las capas de presentación, negocio y datos. Todas las pruebas corren sin conexión al núcleo real.	Revisión de código (sitra.nucleo.modo) + mvn test: las pruebas unitarias corren sin núcleo y sin base de datos.

Nota: "trazabilidad" no es una característica propia de la ISO/IEC 25010. Se ubica dentro de Seguridad, en las sub-características responsabilidad (accountability) y no repudio. La métrica "completitud del registro de auditoría" está definida para esa sub-característica en la ISO/IEC 25023, la norma que dice cómo medir la 25010.

1.4 Restricciones

Ya vienen dadas por el cliente y el regulador; la arquitectura las acata.

Tipo	Restricción	Origen / justificación
Técnica	El núcleo legado no se modifica, ni ahora ni a futuro. Solo se consumen los puntos de acceso que ya expone.	Caso: 20 años, lenguaje propietario, sin documentación, alto riesgo de romper algo al tocarlo. Cliente: no se va a cambiar el sistema.
Técnica	No existe ambiente de pruebas del núcleo; toda prueba previa se hace contra un sustituto controlado.	Consecuencia de la anterior y de la falta de presupuesto: no hay con qué montar una copia del núcleo.
Técnica	Base de datos relacional de licencia libre y con transacciones; la exposición al tercero es una API REST con JSON sobre HTTPS.	Lineamiento del curso + la bitácora exige integridad; interoperabilidad con la billetera.
Operativa (RA-02)	Cero presupuesto: solo software libre e infraestructura que la cooperativa ya tiene. Nada de licencias, nube de pago ni productos de API.	Gerencia: "solo tenemos lo que se tiene"; no se invertirá en APIs.
Operativa (RA-03)	Cero personal técnico: la solución debe operarse por configuración y arrancar sola, sin nadie que la desarrolle o mantenga.	Cliente: el personal solo se dedica a las funciones de la cooperativa; nadie del equipo original.
De tiempo	Plazo regulatorio fijo de 6 meses. En lo académico: corte vertical funcionando el 07/11/2026 y documento final el 21/11/2026.	Regulador (licencia de operación) + cronograma del curso.
Normativa	Trazabilidad auditable y registros no alterables; datos personales tratados según la Ley 1581 de 2012 (habeas data).	Exigencia del auditor + marco colombiano.

Tensión declarada: el regulador obliga a conectar el núcleo con la billetera, pero el cliente no da dinero ni personal. Con literalmente nada nuevo no se puede cumplir. La arquitectura propone el mínimo indispensable —usar primero lo que el núcleo ya tiene y, donde no alcance, un conector pequeño con software gratuito sobre los equipos existentes— y deja en manos de la junta directiva quién lo opera y sostiene. Es la pregunta al cliente de la ficha de drivers.

SECCIÓN 2: ESTRATEGIA ARQUITECTÓNICA Y TECNOLÓGICA

2.1 Patrón arquitectónico principal: N-Capas

Se selecciona el patrón N-Capas (presentación, negocio y datos, más un conector del núcleo) porque el atributo de calidad prioritario AC-03 (Mantenibilidad) exige separar responsabilidades y poder cambiar la conexión con el núcleo por un simulador sin tocar la lógica de negocio, y porque la restricción operativa de cero personal técnico exige una estructura que cualquier desarrollador Java reconozca sin capacitación. Se descarta Hexagonal porque su costo en número de clases, interfaces y curva de aprendizaje contradice esa misma restricción de cero personal técnico; de él solo se toma una idea: el núcleo se usa a través de la interfaz NucleoConector. Se descarta Microservicios porque su costo en complejidad operativa contradice las restricciones de cero presupuesto y cero personal.

Patrón candidato	A favor en este caso	En contra en este caso	Veredicto
MVC monolítico	Muy simple y rápido de construir dentro de los 6 meses.	El controlador terminaría hablando directo con el núcleo y con la base de datos: no hay dónde poner el simulador sin tocar código de negocio.	Descartado: incumple la métrica de AC-03.
N-Capas	Cada capa tiene una sola responsabilidad. Es la estructura estándar de Spring Boot: fácil de leer para quien herede el sistema.	Algo más de código que MVC (servicios y repositorios separados).	SELECCIONADO, con un conector intercambiable para el núcleo (ADR-001).
Hexagonal	Máxima independencia del núcleo y de la base de datos; muy fácil de probar.	Más clases, interfaces y conceptos (puertos, adaptadores); difícil de sostener sin personal técnico.	Descartado como patrón completo; se toma solo la interfaz del conector.
Microservicios	Escala y despliega cada parte por separado.	Varios servicios que operar y monitorear; exige personal y presupuesto que no existen.	Descartado.

Capas y responsabilidades (tal como quedaron en el código)

Capa	Responsabilidad	Clases principales (paquete)
Seguridad	Rechazar toda petición a /api/** sin token válido (401) antes de llegar a los controladores; emitir tokens; comparar secretos con BCrypt.	FiltroJwt, JwtServicio, ConfiguracionSeguridad (security)
Presentación	Recibir la petición HTTP y devolver el código HTTP correcto. No tiene lógica de negocio. Errores centralizados.	AuthController, TransferenciaController, SaldoController, AuditoriaController (controller); ManejadorErrores (exception)
Negocio	Reglas: rol (403), cuenta autorizada, monto positivo, código no repetido, estados de la transferencia, reverso al anular y registro de rechazos en la bitácora.	TransferenciaService, CuentaService, AuditoriaService, IdentificacionService (service)
Datos	Guardar y leer de MariaDB con JPA; consulta nativa de verificación de huellas.	Repositorios (repository) + entidades (model)
Conector del núcleo	Única parte que habla con el núcleo legado. Dos versiones intercambiables por configuración.	NucleoConector (interfaz), NucleoSimuladoConector, NucleoLegadoConector (integration)

Regla de dependencias: cada capa solo llama a la de abajo. El servicio no sabe si habla con el núcleo real o con el simulador; solo conoce la interfaz NucleoConector.

2.2 Pila tecnológica (Tech Stack)

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación (AC / restricción)
Lenguaje backend	Java (OpenJDK)	21 LTS	Node.js 20	Restricción de tiempo: dominio previo del estudiante en Java y Spring Boot. Licencia libre (cero presupuesto). Versión LTS: soporte largo, menos mantenimiento (AC-03).
Framework backend	Spring Boot	3.3.5	Express / Quarkus	AC-03: impone la estructura por capas (@RestController, @Service, @Repository) y permite cambiar el conector por el simulador con una propiedad (sitra.nucleo.modo), sin tocar código.
Seguridad	JWT HS256 firmado con javax.crypto (JDK) + BCrypt (spring-security-crypto)	JDK 21 / 6.3	Sesiones en servidor; servidor de identidad (Keycloak)	ADR-005. AC-01: el consumidor es otro sistema; cada petición prueba quién la hace y ese dato queda en la bitácora. Sin plataformas nuevas (RA-02, RA-03).
Acceso a datos	Spring Data JPA + Hibernate	3.3 / 6.5	JDBC / SQL a mano	AC-03: repositorios sin SQL repetitivo; las entidades corresponden 1 a 1 con las tablas. Se acepta perder control fino del SQL (6.1).
Migraciones	Scripts SQL versionados (V1–V4) ejecutados con HeidiSQL	—	ddl-auto de Hibernate / Flyway	AC-01: el esquema queda versionado y se puede auditar cuándo cambió. HeidiSQL ya está en los equipos (cero personal). Hibernate no crea tablas (ddl-auto=none).
Base de datos	MariaDB (incluida en XAMPP)	10.4+	PostgreSQL 16 / MongoDB	ADR-002. AC-01: llaves foráneas, CHECK, UNIQUE, transacciones InnoDB y triggers que impiden modificar la bitácora. Ya está instalada (RA-02, RA-03).
Huella de integridad	SHA2(..., 256) de MariaDB, en un trigger	10.4+	Servicio de auditoría en la nube	ADR-003. Cero presupuesto: viene con la base de datos. Permite detectar si un registro fue alterado, sin depender del código.
Conector del núcleo	Interfaz Java propia + simulador	—	API Gateway comercial	Cero presupuesto (sin licencias) + no hay ambiente de pruebas del núcleo: el simulador permite probar todo sin tocar producción.
Frontend	Ninguno en el corte vertical	—	SPA en React	Alcance: el consumidor es la billetera (sistema a sistema); el auditor usa la API o HeidiSQL. Postman demuestra el flujo completo.
Pruebas	JUnit 5 + Mockito + Spring Boot Test (MockMvc) + Postman	5.10	Solo pruebas manuales	Verificación de las métricas de AC-01, AC-02 y AC-03 (Sección 5.4).
Despliegue	JAR de Spring Boot (java -jar) + XAMPP existente	—	Docker / nube de pago	Cero presupuesto y cero personal: corre en un equipo que ya existe, sin instalar otra plataforma (Sección 3.5).

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación (AC / restricción)
Diagramas	PlantUML	1.2024	Herramienta con licencia	Los diagramas son texto: se versionan en Git junto al código.
Control de versiones	Git + GitHub	—	—	Restricción del curso.

SECCIÓN 3: VISTAS DE ARQUITECTURA (MODELO C4 Y UML)

Los diagramas están en notación UML y se hicieron con PlantUML. Las fuentes (.puml) y las imágenes en alta resolución están en docs/diagramas/ del repositorio. Todos corresponden al código entregado (nombres de clases, puertos y tecnología).

3.1 Vista de Contexto — C4 Nivel 1

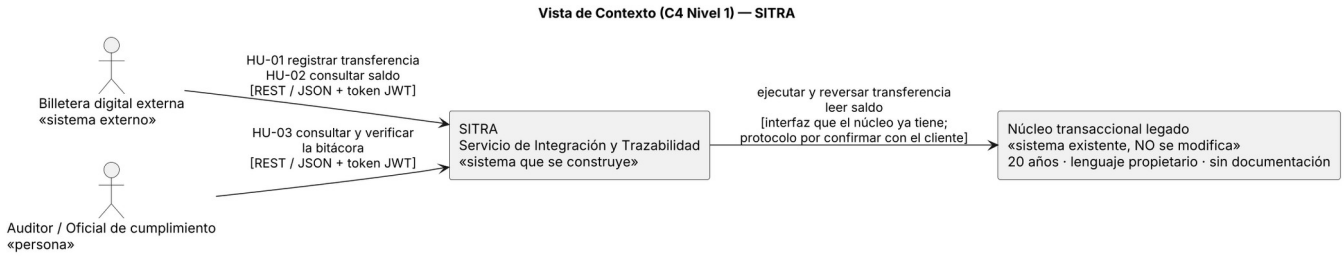


Figura 1. Vista de Contexto: SITRA como caja negra, sus actores y el sistema externo.

Leyenda

- Figura de persona: actor (persona o sistema que inicia una acción). Rectángulo: sistema.
- Texto en las flechas: historia de usuario que viaja por esa relación y, entre corchetes, el protocolo.

Lectura en 30 segundos

SITRA se ubica entre la billetera y el núcleo legado. La billetera solo puede consultar saldos y registrar transferencias (HU-01 y HU-02); el auditor solo consulta y verifica la bitácora (HU-03). Ambos se identifican con un token JWT. Nadie de afuera habla directamente con el núcleo: solo SITRA, y usando la interfaz que el núcleo ya tiene. El protocolo exacto de esa interfaz se marca "por confirmar con el cliente" porque el caso no lo dice; averiguarlo es el primer paso de "trabajar con lo que se tiene".

3.2 Vista de Contenedores / Capas — C4 Nivel 2

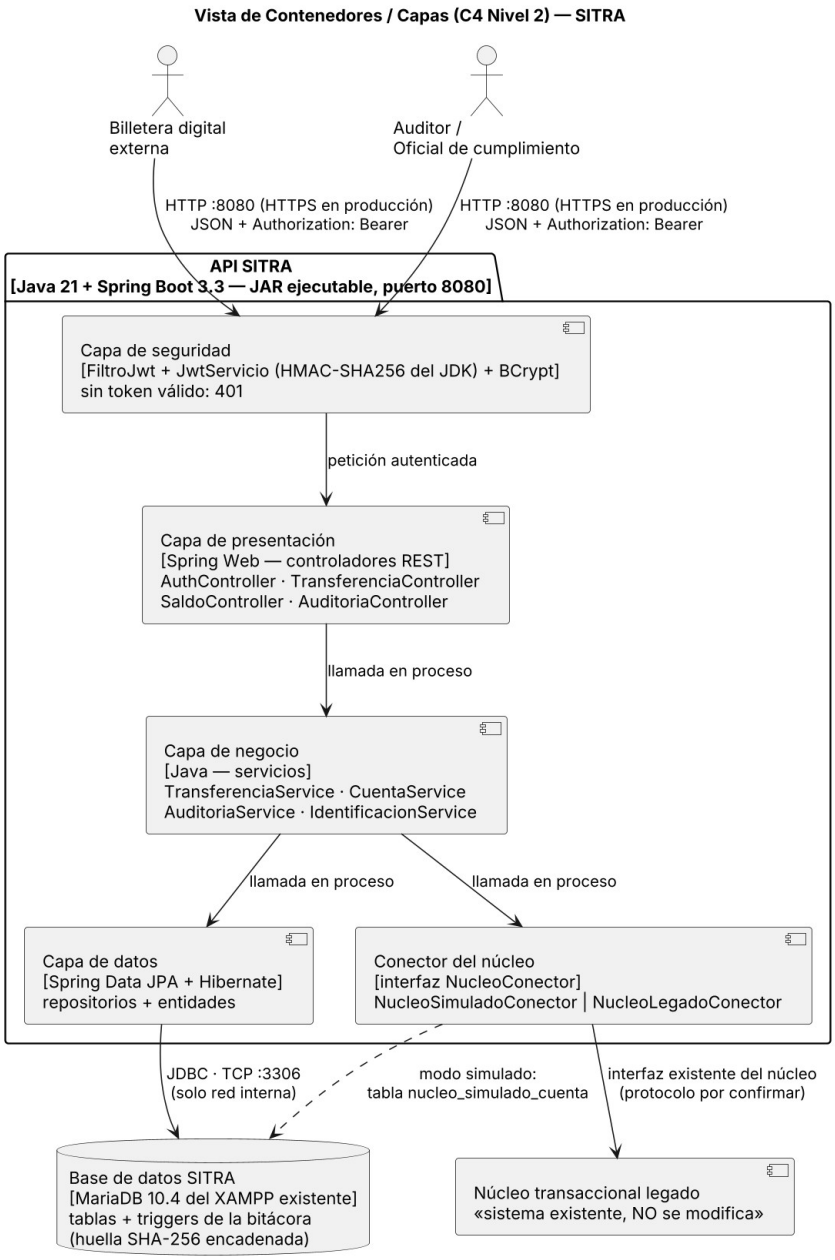


Figura 2. Vista de Contenedores: el contenedor API SITRA con sus capas y la base de datos.

Leyenda

- Marco: contenedor ejecutable (la aplicación). Cajas internas: capas. Cilindro: base de datos. Entre corchetes: tecnología concreta (la misma de la Sección 2.2).
- Etiquetas de las flechas: protocolo y puerto. Línea punteada: solo en modo simulado.

Lectura en 30 segundos

Hay solo dos contenedores: la API SITRA (Java 21 + Spring Boot 3.3, un JAR en el puerto 8080) y la base de datos MariaDB 10.4 del XAMPP existente (puerto 3306, solo red interna), que además tiene los triggers de la bitácora. Dentro de la API, toda petición pasa primero por el filtro JWT; si no trae token válido se responde 401 sin llegar a los controladores. Luego baja por presentación y negocio, y desde negocio a datos o al conector del núcleo. El conector tiene dos versiones —la real y un simulador— y se cambia una por otra con configuración, sin tocar código.

3.3 Vista de Dominio — Diagrama de Clases UML

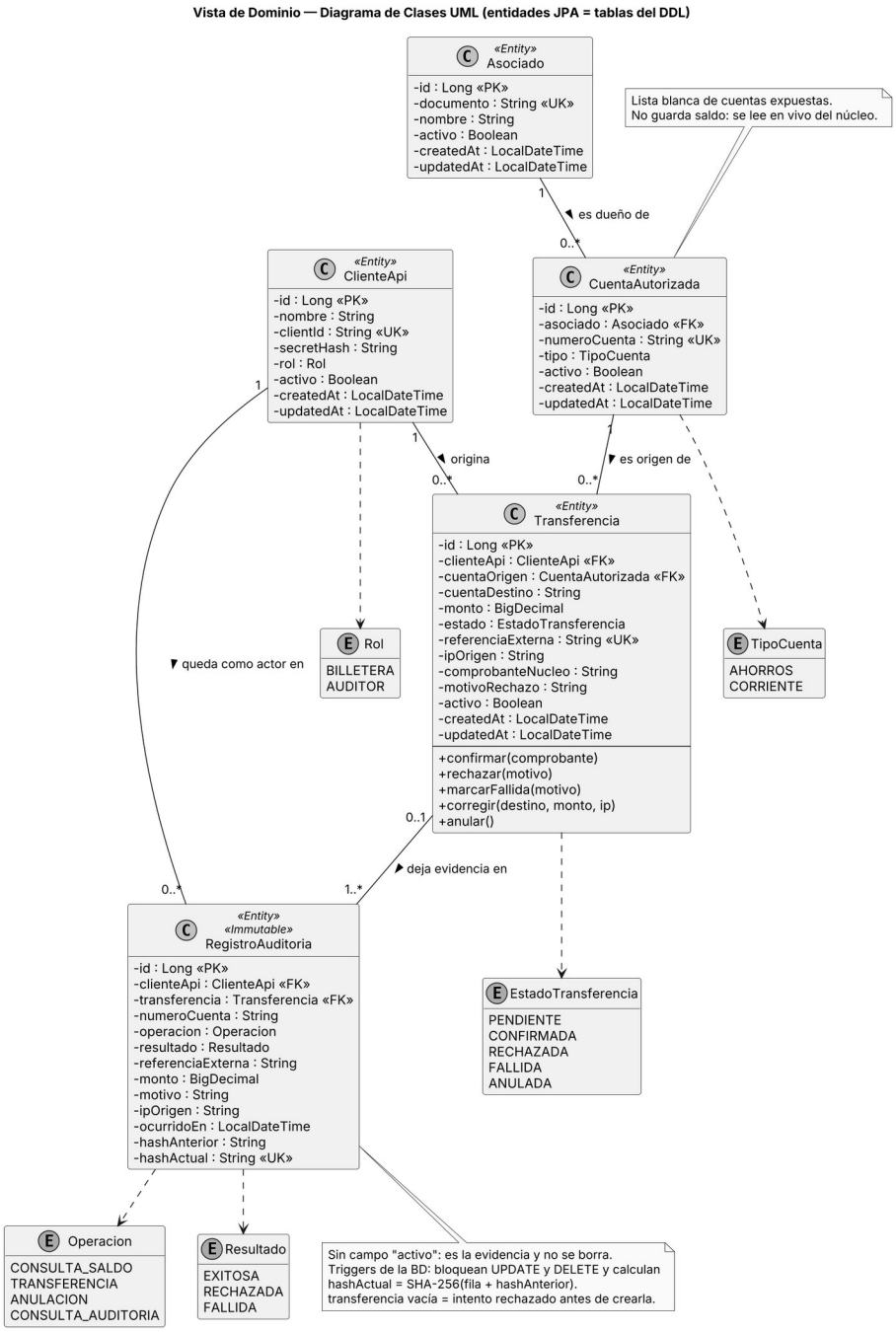


Figura 3. Diagrama de clases: entidades JPA, atributos tipados, llaves y multiplicidades.

Leyenda

- «PK» llave primaria, «FK» llave foránea, «UK» valor único. «Entity» = clase que JPA guarda como tabla; «Immutable» = JPA nunca la actualiza. E = enumeración.
- Números en las líneas: multiplicidad (1 = uno, 0..* = cero o muchos, 0..1 = cero o uno, 1..* = uno o muchos).

Lectura en 30 segundos

Cinco entidades. Transferencia es la principal (HU-01): la origina un ClienteApi (la billetera) desde una CuentaAutorizada, que pertenece a un Asociado; sus métodos son las reglas de estado (confirmar, rechazar, marcarFallida, corregir, anular). Cada transferencia deja uno o más RegistroAuditoria, y cada registro guarda quién,

qué, sobre qué cuenta, con qué resultado, desde qué IP y cuándo. Dos decisiones para defender: CuentaAutorizada no guarda saldo, porque el saldo siempre se lee del núcleo; y RegistroAuditoria no tiene campo activo, porque es la evidencia y no se puede borrar de ninguna forma.

Correspondencia clase ↔ tabla

Clase (Java)	Tabla (MariaDB)	Borrado	Aparece en la secuencia (3.4)
Asociado	asociado	Lógico (activo)	Dueño de la cuenta origen
ClienteApi	cliente_api	Lógico (activo)	Viene en el token (paso 3)
CuentaAutorizada	cuenta_autorizada	Lógico (activo)	Se valida en el servicio (pasos 5–6)
Transferencia	transferencia	Lógico (activo + ANULADA)	INSERT con su estado final (pasos 18–19)
RegistroAuditoria	registro_auditoria	Ninguno (bloqueado por trigger)	Rechazo desde el servicio (7–8) y trigger (20)

3.4 Vista Dinámica — Diagrama de Secuencia UML (HU-01)

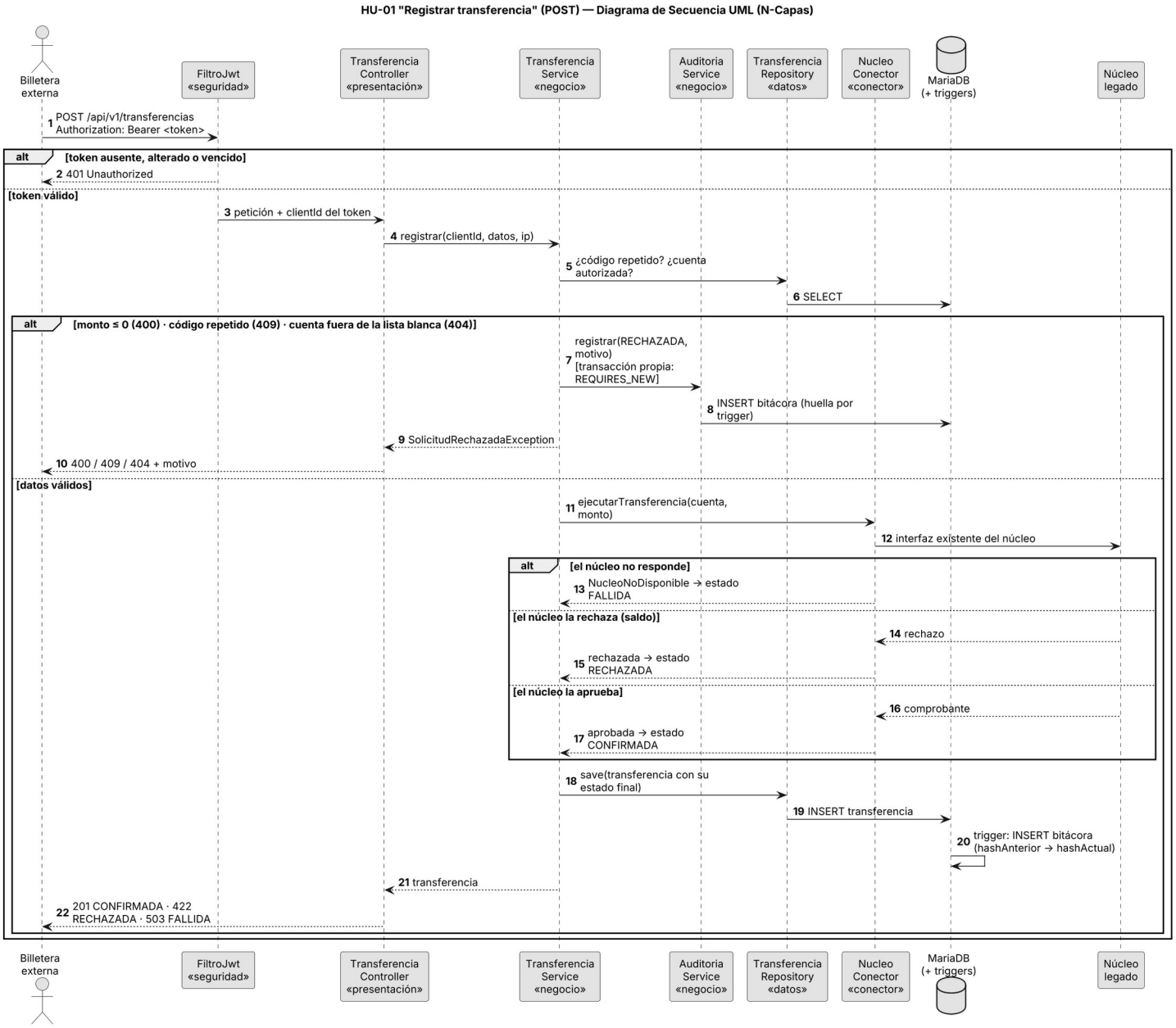


Figura 4. Secuencia de HU-01 (POST): camino feliz y flujos alternos 401, 400/404/409, 422 y 503.

Leyenda: flecha continua = llamada; flecha punteada = respuesta; flecha que vuelve a MariaDB = trigger; recuadro alt = alternativas según una condición. Los pasos están numerados.

Lectura en 30 segundos: sin token válido, el filtro responde 401 y la petición nunca llega al controlador. Si el monto no es válido, el código de operación se repite o la cuenta no está en la lista blanca, la transferencia no se crea: el servicio deja el intento RECHAZADO en la bitácora en una transacción propia (para que no se pierda) y responde 400, 409 o 404. Si los datos son válidos, se llama al núcleo por el conector y la transferencia se guarda una sola vez con su estado final: FALLIDA (503) si el núcleo no responde, RECHAZADA (422) si la rechaza y CONFIRMADA (201) si la aprueba. En los tres casos el trigger de MariaDB escribe la bitácora con su huella: ningún camino queda sin evidencia.

3.5 Vista de Despliegue

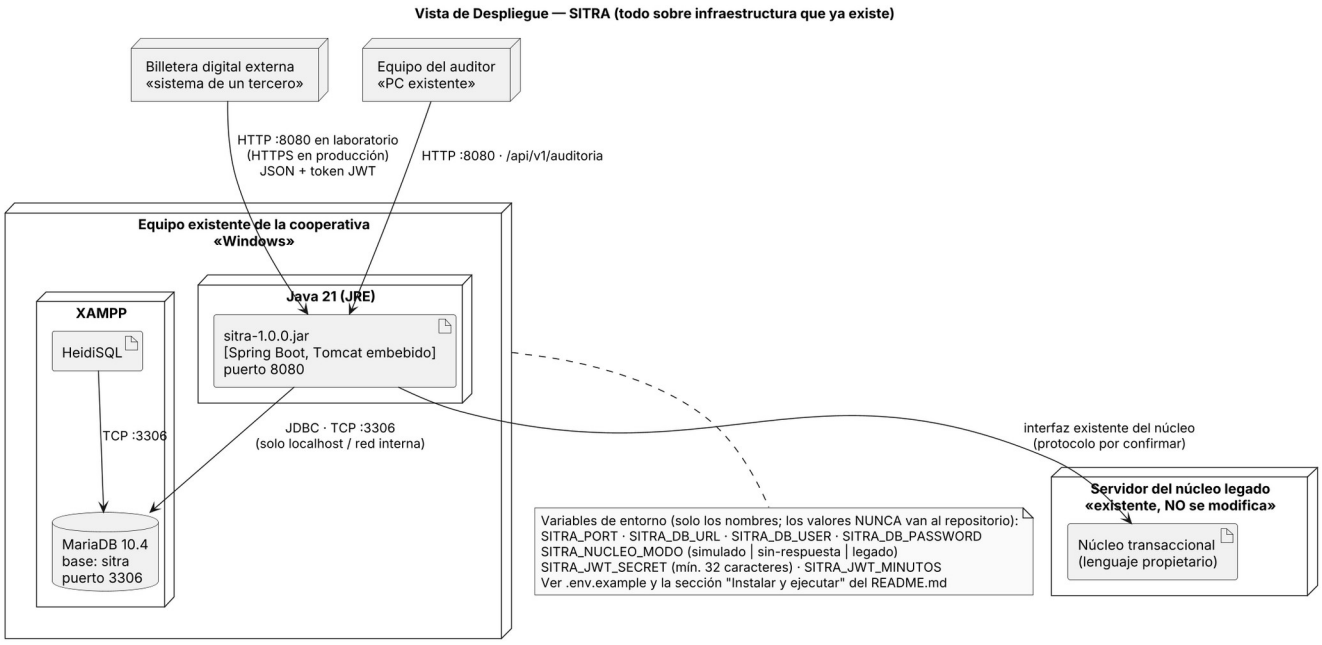


Figura 5. Vista de Despliegue: todo corre en un equipo que la cooperativa ya tiene.

Leyenda

- Caja 3D (nodo): máquina o entorno de ejecución. Hoja con esquina doblada (artefacto): lo que se ejecuta ahí. Cilindro: base de datos.
- Texto de las flechas: protocolo y puerto. La nota lista las variables de entorno sensibles, solo por su nombre.

Lectura en 30 segundos

No se compra ni se instala ninguna plataforma nueva: en un equipo de la cooperativa ya existen XAMPP (con MariaDB y HeidiSQL) y Java. SITRA es un solo archivo JAR que se ejecuta con java -jar y escucha en el puerto 8080; se conecta a MariaDB por el 3306, que solo se usa dentro del equipo o la red interna. La billetera y el auditor entran por el 8080 con su token. El núcleo legado sigue en su propio servidor y solo el conector de SITRA le habla.

Puertos y variables de entorno

Elemento	Valor	Dónde se configura
Puerto de la API	8080 (HTTP en laboratorio; HTTPS en producción)	SITRA_PORT
Base de datos	jdbc:mariadb://localhost:3306/sitra	SITRA_DB_URL
Usuario y contraseña de la BD	solo el nombre de la variable; el valor nunca se versiona (en producción: usuario sitra_app de mínimo privilegio)	SITRA_DB_USER, SITRA_DB_PASSWORD
Modo del conector del núcleo	simulado | sin-respuesta | legado	SITRA_NUCLEO_MODO
Secreto de firma del token	mínimo 32 caracteres; si falta, se genera uno aleatorio al arrancar	SITRA_JWT_SECRET
Vida del token	30 minutos por defecto	SITRA_JWT_MINUTOS

Correspondencia con el repositorio: .env.example (nombres sin valores), src/main/resources/application.properties (lee esas variables) y la sección "Instalar y ejecutar" del README.md.

SECCIÓN 4: REGISTRO DE DECISIONES DE ARQUITECTURA (ADRs)

Cinco decisiones críticas, cada una en docs/adr/ del repositorio con la plantilla obligatoria. Los ADR no se editan: si una decisión cambia, se escribe uno nuevo que la reemplaza (así se hizo con la base de datos en el ADR-002 y con la identificación provisional en el ADR-005). El texto de esta sección se genera desde esos mismos archivos.

ADR	Decisión	Atributo / restricción que la obliga
ADR-001	N-Capas con un conector intercambiable para el núcleo	AC-03 Mantenibilidad; sin ambiente de pruebas del núcleo; cero personal técnico
ADR-002	MariaDB del XAMPP existente en lugar de PostgreSQL	AC-01 integridad; RA-02 cero presupuesto; RA-03 cero personal
ADR-003	La trazabilidad la garantiza la base de datos (triggers) y los rechazados siempre se guardan	AC-01 Seguridad (responsabilidad, no repudio); exigencia del auditor
ADR-004	Actualizar (PUT) es corregir y reintentar lo que no movió dinero; anular pide reverso	AC-01 (la evidencia coincide con lo ocurrido); AC-02 (reintento sin duplicar)
ADR-005	Token JWT firmado con HMAC del JDK y secretos con BCrypt, sin Spring Security completo	AC-01 (401 sin token, responsable en la bitácora); RA-02; RA-03

ADR-001: Patrón N-Capas con un conector intercambiable para el núcleo legado

Estatus

Aceptada

Fecha

2026-10-24

Contexto

SITRA debe exponer consulta de saldo y transferencias de un núcleo legado de 20 años, sin documentación, que no se puede modificar y del que no existe ambiente de pruebas. La cooperativa no tiene personal técnico: quien herede el sistema debe entenderlo sin ayuda.

Atributos en juego: **AC-03 Mantenibilidad** (modularidad y capacidad de ser probado) y **AC-02 Fiabilidad**. Restricciones: núcleo intocable, sin ambiente de pruebas, cero personal técnico, cero presupuesto.

Alternativas consideradas:

- **MVC monolítico:** el controlador hablaría directo con el núcleo y la base de datos.
- **Hexagonal (puertos y adaptadores):** máxima independencia, pero más clases, interfaces y conceptos.
- **Microservicios:** varios servicios que operar y monitorear.
- **N-Capas:** seguridad → presentación → negocio → datos.

Decisión

Decidimos usar N-Capas (seguridad, presentación, negocio y datos) y que el núcleo se use solo a través de la interfaz NucleoConector, con dos implementaciones que se cambian por configuración (sitra.nucleo.modo): NucleoSimuladoConector para pruebas y NucleoLegadoConector para producción, porque AC-03 exige poder probar todo sin el núcleo real y la restricción de cero personal técnico exige la estructura más estándar posible.

Se descarta MVC porque no deja dónde poner el simulador sin tocar código de negocio. Se descarta Hexagonal completo porque su costo en clases y curva de aprendizaje contradice la restricción de cero personal; de él solo se toma la interfaz del conector. Se descarta Microservicios porque exige personal y presupuesto que no existen.

Consecuencias / Trade-offs

- **Positivas:** cualquier desarrollador Java reconoce la estructura (controller, service, repository, model); el servicio no sabe si habla con el núcleo real o con el simulador; cambiar la integración toca una sola clase y un valor de configuración.

- **Negativas:** más código que un MVC simple (servicios y repositorios separados). El dominio no queda totalmente aislado de Spring, como sí lo haría en Hexagonal.
- **Deuda técnica asumida:** NucleoLegadoConector responde "no disponible" porque el protocolo del núcleo está por confirmar con el cliente. Condición para revisarla: cuando el cliente confirme cómo expone sus datos el núcleo (archivo, base de datos o servicio), se implementa solo esa clase, con una espera máxima de 5 s (AC-02).

ADR-002: MariaDB (la del XAMPP existente) en lugar de PostgreSQL

Estatus

Aceptada — reemplaza la elección de PostgreSQL 16 de la Sección 2.2 del DAS v0.4.

Fecha

2026-10-24

Contexto

En el DAS v0.4 se eligió PostgreSQL 16. En la sustentación de la Clase 4 el modelo se probó en vivo en los equipos disponibles, que tienen XAMPP con MariaDB y HeidiSQL ya instalados. La restricción RA-02 dice que solo se trabaja con lo que se tiene y la RA-03 que no hay personal técnico para instalar y mantener otro motor.

Atributos en juego: **AC-01 Seguridad** (integridad de la bitácora), **AC-03 Mantenibilidad**. Restricciones: cero presupuesto, cero personal técnico, usar la infraestructura existente.

Alternativas consideradas:

- **PostgreSQL 16:** muy completo, pero hay que instalarlo y administrarlo aparte.
- **MariaDB 10.4+ del XAMPP:** ya instalado, gratuito, con HeidiSQL para operarlo.
- **H2 en memoria:** no sirve como evidencia porque se borra al apagar.

Decisión

Decidimos usar MariaDB 10.4+ incluida en XAMPP, porque cumple todo lo que exige AC-01 —llaves foráneas, CHECK (MariaDB los aplica desde la 10.2), UNIQUE, transacciones InnoDB y triggers que bloquean la bitácora— y porque ya está instalada, cumpliendo RA-02 y RA-03.

Se descarta PostgreSQL porque, aunque es técnicamente equivalente para este caso, exige instalar y mantener un motor nuevo sin personal que lo haga. Se descarta H2 porque no conserva la evidencia.

Consecuencias / Trade-offs

- **Positivas:** cero instalación; el mismo motor sirve para desarrollo, pruebas y la demostración en clase; HeidiSQL permite al auditor consultar la bitácora sin programar.
- **Negativas:** se pierden funciones propias de PostgreSQL (índices parciales, pgcrypto). Se reemplazan: el índice parcial por uno normal y la huella por SHA2() de MariaDB. Los scripts PostgreSQL de la Clase 4 quedan en el historial de Git, no en la versión actual.
- **Deuda técnica asumida:** XAMPP trae MariaDB 10.4, una versión sin soporte del fabricante. Condición para revisarla: si el sistema pasa a producción real, actualizar a una versión con soporte (10.11 LTS); los scripts no cambian.

ADR-003: La trazabilidad la garantiza la base de datos, no solo el código

Estatus

Aceptada

Fecha

2026-10-24

Contexto

La trazabilidad es el atributo crítico del caso (**AC-01 Seguridad**: responsabilidad y no repudio): el auditor la exige y de ella depende la licencia de operación. En la Clase 4 se comprobó que, si la bitácora la escribe solo la aplicación, una transferencia insertada directamente por SQL no deja rastro. Además el auditor pidió registrar también los intentos rechazados.

Alternativas consideradas:

- **Solo la aplicación escribe la bitácora:** simple, pero cualquier acceso directo a la base de datos (HeidiSQL, otro programa) se salta el registro.
- **Servicio de auditoría en la nube:** con costo recurrente (prohibido por RA-02).
- **Triggers en la base de datos + registro de rechazos desde el servicio.**

Decisión

Decidimos que la base de datos garantice la bitácora con triggers: toda transferencia que se inserte o cambie deja su fila automáticamente; la bitácora no se puede modificar ni borrar; y cada fila guarda una huella SHA-256 encadenada con la anterior. Los intentos rechazados se guardan así: si la base de datos o la validación impiden crear la transferencia, el servicio escribe la fila RECHAZADA en la bitácora en una transacción propia (REQUIRES_NEW); si el núcleo la rechaza (saldo insuficiente), la transferencia se guarda con estado RECHAZADA. Todo con funciones gratuitas de MariaDB.

Consecuencias / Trade-offs

- **Positivas:** ninguna transferencia queda sin evidencia, sin importar por dónde entre; los rechazados nunca se pierden; la regla no depende de que un programador "se acuerde".
- **Negativas:** parte de la lógica vive en la base de datos (triggers y un procedimiento), que es menos visible que el código Java. Cada inserción hace una escritura extra en la bitácora.
- **Deuda técnica asumida:** el procedimiento registrar_transferencia repite en SQL la regla del TransferenciaService, para poder operar y demostrar desde HeidiSQL. Condición para revisarla: si las dos versiones empiezan a diferir, se deja una sola y la otra la llama.

ADR-004: En el CRUD de Transferencia, actualizar (PUT) es "corregir y reintentar"

Estatus

Aceptada

Fecha

2026-11-06

Contexto

La plantilla exige un CRUD completo de la entidad principal de la HU-01 (Transferencia). Pero una transferencia no es un dato cualquiera: si ya movió dinero en el núcleo, cambiar su monto o su destino en SITRA dejaría la evidencia diciendo algo distinto de lo que pasó. Además, en la Clase 5 se identificó un hueco de fiabilidad: si el núcleo no respondía (FALLIDA, 503), la billetera no podía reintentar con el mismo código de operación, porque el código ya existía (409).

Atributos en juego: **AC-01 Seguridad** (la evidencia debe coincidir con lo ocurrido) y **AC-02 Fiabilidad** (poder recuperarse de un fallo del núcleo sin duplicar operaciones).

Alternativas consideradas:

- **PUT libre sobre cualquier campo:** cumple la plantilla, pero permite "maquillar" una transferencia confirmada.
- **No tener PUT:** incumple el CRUD completo.
- **PUT solo sobre lo que no movió dinero (RECHAZADA o FALLIDA):** corregir destino o monto y volver a enviarla al núcleo con el mismo código de operación.

Decisión

Decidimos que PUT /api/v1/transferencias/{id} solo acepte transferencias RECHAZADAS o FALLIDAS: corrige cuentaDestino y monto, conserva el código de operación y vuelve a pasar por el núcleo (200 si queda CONFIRMADA, 422 si el núcleo la vuelve a rechazar, 503 si sigue sin responder). Sobre una CONFIRMADA responde 409. Y DELETE es un borrado lógico (ANULADA, activo = false, 204): si la transferencia ya había movido dinero, primero se pide el reverso al núcleo; si el núcleo no responde, no se anula nada (503).

Consecuencias / Trade-offs

- **Positivas:** el UPDATE tiene sentido de negocio; resuelve el reintento después de un 503 sin romper la idempotencia; la bitácora (trigger de UPDATE) guarda el intento original y el reintento, así que nada se borra ni se maquilla.
- **Negativas:** el PUT no es "genérico" como en el ejemplo de la plantilla; quien lo use debe conocer la regla (está en el README y la colección Postman la prueba: 409 y 200).
- **Deuda técnica asumida:** el reverso depende de que el núcleo real ofrezca una operación para devolver dinero; hoy solo existe en el simulador. Condición para revisarla: al implementar NucleoLegadoConector (ADR-001), confirmar con el cliente cómo se reversa una operación.

ADR-005: Token JWT firmado con HMAC del JDK y secretos con BCrypt, sin Spring Security completo

Estatus

Aceptada — reemplaza la identificación provisional por encabezado X-Cliente-Id del arranque (Clase 5).

Fecha

2026-11-06

Contexto

El consumidor de SITRA es otro sistema (la billetera), no una persona con navegador. Cada petición debe probar quién la hace, porque ese dato queda en la bitácora como responsable (**AC-01**: responsabilidad y no repudio; meta: 401 en el 100 % de los intentos sin token). En el arranque se usó el encabezado X-Cliente-Id, que cualquiera puede falsificar.

Restricciones: cero presupuesto (RA-02) y cero personal técnico (RA-03): no hay quién instale ni opere un servidor de identidad.

Alternativas consideradas:

- **Sesiones en servidor:** pensadas para navegadores; no aplican a un sistema que llama a otro.
- **Servidor de identidad (por ejemplo Keycloak, OAuth2):** muy completo, pero es otra plataforma que instalar, actualizar y operar.
- **Spring Security completo + librería JWT externa:** estándar, pero agrega una cadena de filtros y configuración que un mantenedor sin experiencia difícilmente entiende.
- **Token JWT HS256 firmado con javax.crypto (incluido en Java) + BCrypt de spring-security-crypto + un filtro propio.**

Decisión

Decidimos usar JWT HS256 firmado con HMAC-SHA256 del propio JDK (JwtServicio), validado en un solo filtro (FiltroJwt) antes de llegar a los controladores, y guardar los secretos de los clientes solo como hash BCrypt (spring-security-crypto, sin el resto de Spring Security), porque AC-01 exige identificar a quien llama en cada petición y RA-02/RA-03 exigen no agregar plataformas ni productos. El secreto de firma llega por la variable SITRA_JWT_SECRET (mínimo 32 caracteres); si no existe, se genera uno aleatorio al arrancar. El token vence a los 30 minutos.

Consecuencias / Trade-offs

- **Positivas:** 401 sin token en todas las rutas /api/** (salvo /api/v1/auth/token); el rol viaja firmado y no se puede cambiar sin invalidar la firma (probado en JwtServicioTest); los secretos nunca se guardan en texto plano; no hay nada nuevo que instalar.
- **Negativas:** código de seguridad propio (unas 240 líneas en el paquete security) que hay que cuidar: comparación de firmas en tiempo constante, se rechaza cualquier algoritmo distinto de HS256 y se valida el vencimiento. No hay renovación de tokens (refresh) ni lista de revocación; se mitiga porque en cada petición se verifica que el cliente siga activo en la base de datos: desactivarlo bloquea sus tokens de inmediato.
- **Deuda técnica asumida:** HTTPS. En el laboratorio la API corre en HTTP (puerto 8080) y el token viaja sin cifrar. Condición para revisarla: antes de producción se activa HTTPS en Spring Boot con un certificado gratuito y se rota SITRA_JWT_SECRET.

SECCIÓN 5: ENTREGABLE DE PROGRAMACIÓN EJECUTABLE (CORTE VERTICAL)

El corte vertical implementa el CRUD completo de Transferencia, la entidad principal de la HU-01, atravesando todas las capas: API REST → filtro JWT → controlador → servicio → repositorio/ORM → MariaDB (con triggers) → y de regreso. Además expone la consulta de saldo (HU-02) y la consulta y verificación de la bitácora (HU-03).

5.1 Repositorio Git

```
sitra/
├── docs/            DAS.docx, DAS.pdf, adr/ (ADR-001 a ADR-005), diagramas/ (.puml + .png)
├── db/
│   ├── migrations/  V1 tablas · V2 triggers de bitácora · V3 rechazados · V4 IP de origen
│   ├── datos_prueba.sql   clientes (secretos BCrypt), asociados, cuentas y saldos
│   ├── instalar_todo.sql  todo lo anterior en un solo archivo (F9 en HeidiSQL)
│   ├── pruebas/demo_sustentacion.sql   demostración paso a paso en HeidiSQL
│   └── produccion/usuario_minimo_privilegio.sql
├── src/main/java/co/coop/sitra/
│   ├── security/     FiltroJwt, JwtServicio, ConfiguracionSeguridad
│   ├── controller/   Auth, Transferencia, Saldo y Auditoria Controller
│   ├── service/      Transferencia, Cuenta, Auditoria e Identificacion Service
│   ├── repository/   repositorios Spring Data JPA
│   ├── model/        entidades JPA (= tablas) y enumeraciones
│   ├── integration/  NucleoConector + simulador + conector legado
│   ├── dto/          cuerpos JSON de entrada y salida
│   └── exception/    ManejadorErrores: errores → códigos HTTP
├── src/main/resources/application.properties   lee las variables de entorno
├── src/test/java/co/coop/sitra/   pruebas unitarias e integración (5.4)
├── postman/coleccion-corte-vertical.json        19 peticiones con tests
├── evidencias/      una carpeta por clase
├── .env.example  ·  .gitignore  ·  pom.xml  ·  README.md
```

Política de commits: commits atómicos con formato Conventional Commits (feat:, fix:, docs:, test:, chore:). El trabajo de cada clase se hace en una rama feature/* que se une a main al terminar, así main siempre queda ejecutable.
El .gitignore excluye .env, target/ y los archivos del editor: el archivo .env con valores reales nunca entra al historial.

5.2 Script SQL DDL / Migraciones (MariaDB)

Cuatro scripts en orden. V1 crea las tablas; V2 los triggers de la bitácora; V3 el procedimiento que registra también los rechazados; V4 es el ALTER que agrega la IP de origen a la transacción (cambio pedido en la Clase 4).
db/instalar_todo.sql los ejecuta todos de una vez junto con los datos de prueba. A continuación, V1 completo:

```
-- ================================================================
-- SITRA — V1: tablas, llaves, CHECK, UNIQUE e índices
-- Motor: MariaDB 10.4+ (XAMPP). Ejecutar en orden: V1, V2, V3, V4.
-- ================================================================

-- Quién usa SITRA: la billetera externa y el auditor
CREATE TABLE cliente_api (
    id           BIGINT        NOT NULL AUTO_INCREMENT PRIMARY KEY,
    nombre       VARCHAR(120)  NOT NULL,
    client_id    VARCHAR(80)   NOT NULL UNIQUE,
    secret_hash  VARCHAR(255)  NOT NULL,                          -- BCrypt, nunca el secreto
    rol          VARCHAR(20)   NOT NULL DEFAULT 'BILLETERA',
    activo       BOOLEAN       NOT NULL DEFAULT TRUE,             -- borrado lógico
    created_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT ck_cliente_rol CHECK (rol IN ('BILLETERA', 'AUDITOR'))
```

```
) ENGINE = InnoDB;

-- Asociado de la cooperativa
CREATE TABLE asociado (
    id          BIGINT        NOT NULL AUTO_INCREMENT PRIMARY KEY,
    documento   VARCHAR(30)   NOT NULL UNIQUE,
    nombre      VARCHAR(120)  NOT NULL,
    activo      BOOLEAN       NOT NULL DEFAULT TRUE,                  -- borrado lógico
    created_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE = InnoDB;

-- Cuenta (lista blanca de cuentas expuestas). NO guarda saldo: el saldo vive en el núcleo.
CREATE TABLE cuenta_autorizada (
    id             BIGINT       NOT NULL AUTO_INCREMENT PRIMARY KEY,
    asociado_id    BIGINT       NOT NULL,
    numero_cuenta  VARCHAR(30)  NOT NULL UNIQUE,
    tipo           VARCHAR(20)  NOT NULL,
    activo         BOOLEAN      NOT NULL DEFAULT TRUE,                -- borrado lógico
    created_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_cuenta_asociado FOREIGN KEY (asociado_id) REFERENCES asociado(id),
    CONSTRAINT ck_cuenta_tipo CHECK (tipo IN ('AHORROS', 'CORRIENTE'))
) ENGINE = InnoDB;

-- Transacción (transferencia)
CREATE TABLE transferencia (
    id                  BIGINT         NOT NULL AUTO_INCREMENT PRIMARY KEY,
    cliente_api_id      BIGINT         NOT NULL,
    cuenta_origen_id    BIGINT         NOT NULL,
    cuenta_destino      VARCHAR(30)    NOT NULL,
    monto               DECIMAL(14,2)  NOT NULL,
    estado              VARCHAR(20)    NOT NULL DEFAULT 'PENDIENTE',
    referencia_externa  VARCHAR(80)    NOT NULL,                       -- código de operación
    comprobante_nucleo  VARCHAR(80)    NULL,
    motivo_rechazo      VARCHAR(255)   NULL,                           -- por qué se rechazó
    activo              BOOLEAN        NOT NULL DEFAULT TRUE,          -- borrado lógico
    created_at          DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at          DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_transf_cliente FOREIGN KEY (cliente_api_id)   REFERENCES cliente_api(id),
    CONSTRAINT fk_transf_cuenta  FOREIGN KEY (cuenta_origen_id) REFERENCES cuenta_autorizada(id),
    CONSTRAINT uq_transf_idempotencia UNIQUE (referencia_externa),    -- llave de idempotencia
    CONSTRAINT ck_transf_monto  CHECK (monto > 0),                    -- CHECK de monto > 0
    CONSTRAINT ck_transf_estado CHECK (estado IN ('PENDIENTE','CONFIRMADA','RECHAZADA','FALLIDA','ANULADA'))
) ENGINE = InnoDB;

-- Bitácora de auditoría (obligatoria: la trazabilidad es el atributo crítico).
-- Guarda también los intentos RECHAZADOS: código, monto intentado y motivo.
-- monto NO tiene CHECK: debe poder guardar lo que se intentó, aunque sea inválido (ej. -50.000).
CREATE TABLE registro_auditoria (
    id                  BIGINT         NOT NULL AUTO_INCREMENT PRIMARY KEY,
    cliente_api_id      BIGINT         NOT NULL,                      -- quién
    transferencia_id    BIGINT         NULL,                          -- vacío si la transferencia no se pudo crear
    numero_cuenta       VARCHAR(30)    NOT NULL,                      -- sobre qué cuenta
    operacion           VARCHAR(30)    NOT NULL,                      -- qué
    resultado           VARCHAR(20)    NOT NULL,                      -- EXITOSA / RECHAZADA / FALLIDA
    referencia_externa  VARCHAR(80)    NULL,                          -- código de operación intentado
```

```
    monto              DECIMAL(14,2)  NULL,                          -- monto intentado
    motivo             VARCHAR(255)   NULL,                          -- por qué se rechazó
    ip_origen          VARCHAR(45)    NOT NULL DEFAULT 'local',
    ocurrido_en        DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,  -- cuándo
    hash_anterior      CHAR(64)       NULL,
    hash_actual        CHAR(64)       NULL UNIQUE,
    CONSTRAINT fk_audit_cliente FOREIGN KEY (cliente_api_id)   REFERENCES cliente_api(id),
    CONSTRAINT fk_audit_transf  FOREIGN KEY (transferencia_id) REFERENCES transferencia(id),
    CONSTRAINT ck_audit_operacion CHECK (operacion IN
('CONSULTA_SALDO','TRANSFERENCIA','ANULACION','CONSULTA_AUDITORIA')),
    CONSTRAINT ck_audit_resultado CHECK (resultado IN ('EXITOSA','RECHAZADA','FALLIDA'))
) ENGINE = InnoDB;

-- Simulador del núcleo legado: aquí vive el SALDO (SITRA no guarda saldos)
CREATE TABLE nucleo_simulado_cuenta (
    numero_cuenta  VARCHAR(30)    NOT NULL PRIMARY KEY,
    saldo          DECIMAL(14,2)  NOT NULL,
    CONSTRAINT ck_nucleo_saldo CHECK (saldo >= 0)
) ENGINE = InnoDB;

-- Índices justificados
CREATE INDEX idx_transferencia_cuenta   ON transferencia (cuenta_origen_id);
CREATE INDEX idx_auditoria_cuenta_fecha ON registro_auditoria (numero_cuenta, ocurrido_en);
```

db/migrations/V4__ip_origen_en_transferencia.sql (extracto)

```
ALTER TABLE transferencia
    ADD COLUMN ip_origen VARCHAR(45) NULL AFTER referencia_externa;
```

Cumplimiento de los requisitos evaluables

Requisito de la plantilla	Dónde se cumple
PK en toda tabla	id BIGINT AUTO_INCREMENT PRIMARY KEY en 5 tablas; nucleo_simulado_cuenta usa numero_cuenta.
FK con REFERENCES	cuenta_autorizada → asociado; transferencia → cliente_api y cuenta_autorizada; registro_auditoria → cliente_api y transferencia.
Al menos una CHECK	monto > 0 (ck_transf_monto), estado, rol, tipo de cuenta, operacion, resultado y saldo ≥ 0.
Al menos una UNIQUE	referencia_externa = llave de idempotencia (uq_transf_idempotencia); client_id, documento, numero_cuenta, hash_actual.
Borrado lógico	Campo activo en asociado, cliente_api, cuenta_autorizada y transferencia (DELETE de la API = ANULADA + activo = false). La bitácora no lo tiene a propósito: es evidencia.
Índices justificados	idx_transferencia_cuenta (historial por cuenta) e idx_auditoria_cuenta_fecha (HU-03: el auditor consulta por cuenta y fechas, meta ≤ 5 s).

Verificación ejecutada en MariaDB (db/instalar_todo.sql + db/pruebas/demo_sustentacion.sql)

```
Paso 1  transferencia 300.000 por SQL -> bitácora: TRANSFERENCIA · EXITOSA (la escribió el trigger)
Paso 2  mismo código                  -> ERROR 1062 Duplicate entry ... 'uq_transf_idempotencia'
        monto -50.000                 -> ERROR 4025 CONSTRAINT 'ck_transf_monto' failed
Paso 4  CALL con código repetido      -> RECHAZADA · queda en la bitácora con su motivo
        CALL con -50.000              -> RECHAZADA · queda en la bitácora con su motivo
        CALL con 5.000.000            -> RECHAZADA · transacción con estado RECHAZADA + bitácora
```

```
Paso 5  verificación de la cadena    -> cadena OK · huella OK en todas las filas
        UPDATE / DELETE en bitácora  -> ERROR 1644: no se puede modificar / borrar
Usuario sitra_app (mínimo privilegio): DROP TRIGGER -> ERROR 1142 · DELETE -> ERROR 1142
Alteración simulada (sin triggers)   -> fila modificada: ALTERADO · fila borrada: la siguiente ROTA
```

5.3 Código backend por capas (Spring Boot + JPA)

Lo evaluable es la separación de capas. Cada petición entra por el filtro JWT; el controlador solo traduce a HTTP; el servicio aplica las reglas y es el único que conoce el conector; el repositorio habla con MariaDB.

Endpoints

Método y ruta	Qué hace	Respuestas
POST /api/v1/auth/token	Cambia clientId + secreto (comparado con BCrypt) por un token JWT de 30 min	200 · 401
POST /api/v1/transferencias	CREATE (HU-01): valida, pasa por el núcleo y guarda con su estado final	201 · 400 · 401 · 403 · 404 · 409 · 422 · 503
GET /api/v1/transferencias	READ: lista las transferencias activas del cliente	200 · 401 · 403
GET /api/v1/transferencias/{id}	READ: consulta una transferencia activa propia	200 · 401 · 404
PUT /api/v1/transferencias/{id}	UPDATE (ADR-004): corrige y reintenta una RECHAZADA o FALLIDA	200 · 400 · 401 · 404 · 409 · 422 · 503
DELETE /api/v1/transferencias/{id}	DELETE lógico: reverso en el núcleo si movió dinero; ANULADA + activo = false	204 · 401 · 404 · 409 · 503
GET /api/v1/cuentas/{numero}/saldo	HU-02: saldo leído en vivo del núcleo (queda CONSULTA_SALDO en la bitácora)	200 · 401 · 403 · 404 · 503
GET /api/v1/auditoria?cuenta=&desde=&hasta=	HU-03: bitácora por cuenta y rango de fechas (solo AUDITOR)	200 · 400 · 401 · 403
GET /api/v1/auditoria/verificacion	HU-03: recalcula la cadena de huellas; integra = true si nadie alteró nada	200 · 401 · 403

Controlador — src/main/java/co/coop/sitra/controller/TransferenciaController.java (extracto)

```
/** CREATE: 201 CONFIRMADA · 422 RECHAZADA por el núcleo · 503 FALLIDA (núcleo sin respuesta). */
@PostMapping
public ResponseEntity<TransferenciaResponse> registrar(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
                                                       @RequestBody CrearTransferenciaRequest datos,
                                                       HttpServletRequest peticion) {
    Transferencia t = servicio.registrar(clientId, datos, peticion.getRemoteAddr());
    return ResponseEntity.status(codigoSegunEstado(t, HttpStatus.CREATED)).body(TransferenciaResponse.de(t));
}

/** READ: solo las activas del cliente que pregunta. */
@GetMapping
public List<TransferenciaResponse> listar(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId) {
    return servicio.listar(clientId).stream().map(TransferenciaResponse::de).toList();
}

@GetMapping("/{id}")
public TransferenciaResponse obtener(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
                                     @PathVariable Long id) {
    return TransferenciaResponse.de(servicio.obtener(clientId, id));
```

```
}

/** UPDATE: corregir y reintentar una RECHAZADA o FALLIDA (ADR-004). 200 · 422 · 503 · 409. */
@PutMapping("/{id}")
public ResponseEntity<TransferenciaResponse> actualizar(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
                                                        @PathVariable Long id,
                                                        @RequestBody ActualizarTransferenciaRequest datos,
                                                        HttpServletRequest peticion) {
    Transferencia t = servicio.actualizar(clientId, id, datos, peticion.getRemoteAddr());
    return ResponseEntity.status(codigoSegunEstado(t, HttpStatus.OK)).body(TransferenciaResponse.de(t));
}

/** DELETE lógico: 204 y la fila queda con activo = false y estado ANULADA. */
@DeleteMapping("/{id}")
public ResponseEntity<Void> anular(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
                                   @PathVariable Long id, HttpServletRequest peticion) {
    servicio.anular(clientId, id, peticion.getRemoteAddr());
    return ResponseEntity.noContent().build();
}
```

Servicio — src/main/java/co/coop/sitra/service/TransferenciaService.java (CREATE, extracto)

```
@Transactional
public Transferencia registrar(String clientId, CrearTransferenciaRequest datos, String ip) {
    ClienteApi cliente = identificacion.exigirRol(clientId, Rol.BILLETERA);
    String cuenta = datos.cuentaOrigen();
    if (vacio(cuenta) || vacio(datos.cuentaDestino()) || vacio(datos.referenciaExterna()) || datos.monto() == null) {
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.BAD_REQUEST, "Faltan datos: cuentaOrigen, cuentaDestino, monto y referenciaExterna son obligatorios",
ip);
    }
    if (datos.monto().compareTo(BigDecimal.ZERO) <= 0) {
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.BAD_REQUEST, "El monto debe ser mayor que cero", ip);
    }
    if (transferencias.existsByReferenciaExterna(datos.referenciaExterna())) {
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.CONFLICT, "El código de operación ya existe: la transferencia no se repite", ip);
    }
    CuentaAutorizada origen = cuentas.findByNumeroCuentaAndActivoTrue(cuenta).orElse(null);
    if (origen == null) {
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.NOT_FOUND, "La cuenta no está en la lista de cuentas autorizadas", ip);
    }

    Transferencia t = Transferencia.nueva(cliente, origen, datos.cuentaDestino(), datos.monto(),
            datos.referenciaExterna(), ip);
    ejecutarEnNucleo(t);
    try {
        return transferencias.save(t);   // INSERT: el trigger escribe la bitácora
    } catch (DataIntegrityViolationException e) {
        // Dos peticiones iguales al mismo tiempo: la llave UNIQUE de la base de datos frena la segunda
        throw rechazo(cliente, null, cuenta, Operacion.TRANSFERENCIA, datos.referenciaExterna(), datos.monto(),
                HttpStatus.CONFLICT, "La base de datos rechazó la operación (código de operación repetido)", ip);
    }
}
```

Cómo se ejecuta (detalle en el README.md)

- XAMPP Control Panel → Start en MySQL.
- HeidiSQL → abrir db/instalar_todo.sql → F9 (crea la base sitra con datos de prueba).
- VS Code → abrir la carpeta del repositorio → Run sobre SitraApplication.java (o mvn spring-boot:run). La API queda en http://localhost:8080.
- Postman → importar postman/coleccion-corte-vertical.json → Run collection (19 peticiones en orden).
- Pruebas automáticas: mvn test (unitarias, sin base de datos) y mvn test -Pintegracion (con MariaDB).

5.4 Pruebas de la arquitectura

Se entregan las dos modalidades que acepta la plantilla.

a) Colección Postman (postman/coleccion-corte-vertical.json) — 19 peticiones, todas con tests automáticos

#	Petición	Esperado	Qué valida
01	POST /auth/token (billetera)	200 + token	Autenticación con BCrypt
02	POST /auth/token con secreto incorrecto	401	AC-01
03	POST /transferencias sin token	401	AC-01: nunca llega al controlador
04	POST /transferencias 300.000	201 CONFIRMADA	CREATE por todas las capas
05	POST con el mismo código	409	Idempotencia (AC-02)
06	POST monto −50.000	400	Regla de negocio; queda RECHAZADO en bitácora
07	POST cuenta AH-9999	404	Lista blanca (mínimo privilegio)
08	GET /cuentas/AH-1001/saldo	200	HU-02, saldo del núcleo
09–10	GET lista y GET por id	200	READ: la creada aparece
11	PUT sobre la CONFIRMADA	409	ADR-004: no se maquilla lo que movió dinero
12	POST 5.000.000	422 RECHAZADA	Rechazo del núcleo
13	PUT de la RECHAZADA con 100.000	200 CONFIRMADA	UPDATE: corregir y reintentar
14–15	DELETE de la de 300.000 y GET por id	204 y 404	DELETE lógico con reverso
16–17	Token del auditor; bitácora con token de billetera	200 y 403	Autorización por rol
18	GET /auditoria?cuenta=AH-1001&desde=hoy&hasta=hoy	200	HU-03 y métrica X = A/B: creada, rechazos, corrección y anulación registrados
19	GET /auditoria/verificacion	200, integra = true	Nadie alteró ni borró filas

b) Pruebas JUnit 5 (src/test/java)

Clase de prueba	Tipo	Qué comprueba	Cómo se corre
JwtServicioTest (6 pruebas)	Unitaria	Token válido; token alterado (cambio de rol), con otro secreto, vencido o mal formado → rechazado; secreto corto → no arranca.	mvn test
TransferenciaServiceTest (11 pruebas)	Unitaria con Mockito	201/400/404/409/422/503 en CREATE; FALLIDA sin efecto parcial (AC-02); PUT 409 sobre CONFIRMADA y reintento de RECHAZADA; DELETE con reverso y 503 sin anular. Sin núcleo ni base de datos (AC-03).	mvn test
CorteVerticalIntegracionTest (4 pruebas)	Integración (MockMvc + MariaDB)	401 sin token; CRUD completo punta a punta; RECHAZADA → PUT → CONFIRMADA; saldo, 403 de la billetera en la bitácora y verificación íntegra.	mvn test -Pintegracion

Criterio de aceptación: desde una terminal limpia, siguiendo solo el README.md, se instala la base (F9 en HeidiSQL), se arranca la API y se ejecutan las pruebas (mvn test, mvn test -Pintegracion y el Runner de Postman) sin intervención del autor. Las capturas de esas ejecuciones se guardan en evidencias/entrega-07-pruebas-y-tradeoffs/.

SECCIÓN 6: EVALUACIÓN DE ARQUITECTURA Y TRADE-OFFS

6.1 Análisis de compromisos

Toda arquitectura sacrifica algo. Este es el balance de SITRA:

Atributo priorizado	Atributo sacrificado	Evidencia del compromiso	¿Por qué es aceptable en este proyecto?
Seguridad / trazabilidad (AC-01)	Eficiencia de desempeño (escrituras concurrentes)	Cada operación escribe una fila extra en la bitácora y el trigger lee la última huella. Medido en MariaDB: 6 inserciones simultáneas → cadena íntegra, pero 4 terminaron en deadlock (error 1213).	El volumen de una cooperativa es bajo; la operación que pierde el bloqueo se deshace completa y responde 503: la billetera reintenta con el mismo código sin duplicar (R-03).
Mantenibilidad (AC-03)	Rendimiento en consultas	JPA carga ClienteApi y Transferencia de cada fila de la bitácora (consultas N+1) al listarla.	La consulta del auditor siempre va por cuenta y fechas (índice idx_auditoria_cuenta_fecha); el volumen mantiene la meta de ≤ 5 s. Si crece: consulta nativa puntual.
Seguridad (AC-01)	Simplicidad / velocidad de desarrollo	Filtro JWT, firma HMAC y BCrypt agregan unas 240 líneas de código propio y un paso más en cada petición.	Se mueve dinero y datos personales (Ley 1581); se evita instalar un servidor de identidad (ADR-005) y el código está cubierto por 6 pruebas.
Integridad de la evidencia (AC-01)	Flexibilidad del CRUD	PUT solo sobre RECHAZADA o FALLIDA; DELETE exige reverso en el núcleo (ADR-004).	Una transferencia confirmada no puede "maquillarse": la bitácora siempre coincide con lo que pasó en el núcleo.
Cero presupuesto (RA-02) y cero personal (RA-03)	Disponibilidad	Todo corre en un solo equipo existente, sin réplicas ni balanceo (Sección 3.5).	La alta disponibilidad está fuera del alcance (1.1); lo crítico es no perder la bitácora: respaldo diario con mysqldump, que trae XAMPP.

6.2 Puntos de sensibilidad y riesgos técnicos

Un punto de sensibilidad es un elemento de la arquitectura cuyo cambio impacta directamente un atributo de calidad.

#	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Prob. / Impacto	Mitigación propuesta
R-01	Secreto de firma del token (SITRA_JWT_SECRET)	Seguridad	Si se filtra, cualquiera fabrica tokens y suplanta a la billetera o al auditor.	Baja / Crítico	Variable de entorno fuera de Git (.env en .gitignore); mínimo 32 caracteres; aleatorio si falta; token de 30 min; rotación antes de producción.
R-02	Orden "primero el núcleo, después guardar" en TransferenciaService	Fiabilidad / integridad	El núcleo descuenta pero SITRA falla al guardar: dinero movido sin transferencia registrada.	Baja / Alto	Con el simulador, la misma transacción lo deshace. Con el núcleo real: enviar el código de operación al núcleo y conciliar a diario

#	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Prob. / Impacto	Mitigación propuesta
					contra sus reportes; evaluar guardar PENDIENTE antes de llamar.
R-03	Trigger de huella encadenada (lee la última fila de la bitácora)	Fiabilidad / eficiencia	Con escrituras simultáneas, InnoDB bloquea y aparecen deadlocks (medido: 4 de 6).	Media si crece el volumen / Medio	ManejadorErrores responde 503 reintentable; la idempotencia evita duplicados; si el volumen crece: reintento automático o una sola cola de escritura.
R-04	Credenciales de la base (root sin contraseña en XAMPP)	Seguridad	Alguien con acceso a la base desactiva los triggers y altera la bitácora.	Media / Crítico	Usuario sitra_app sin DELETE ni TRIGGER (probado: error 1142); la verificación de huellas detecta cualquier cambio (probado: ALTERADO / ROTA).
R-05	Transporte HTTP en el puerto 8080	Seguridad (confidencialidad)	Token y datos viajan sin cifrar en la red.	Alta en laboratorio / Alto	HTTPS en Spring Boot con certificado gratuito antes de producción (deuda del ADR-005).
R-06	Protocolo del núcleo sin confirmar (NucleoLegadoConector)	Adecuación funcional	No poder integrar el núcleo real dentro de los 6 meses.	Media / Crítico	Conector aislado (ADR-001): solo cambia una clase; es la primera tarea con el cliente.
R-07	MariaDB 10.4 de XAMPP sin soporte	Seguridad	Vulnerabilidades sin parche.	Media / Medio	Actualizar a MariaDB 10.11 LTS; los scripts ya se ejecutaron en 10.11 sin cambios.

6.3 Conclusión arquitectónica

La arquitectura entregada satisface los tres escenarios de la Sección 1.3 con evidencia del corte vertical. AC-01: sin token se responde 401 antes de llegar a los controladores (filtro JWT, peticiones 02–03 y pruebas JUnit); el 100 % de las operaciones, incluidas las rechazadas, quedan en la bitácora porque la base de datos la escribe con triggers y el servicio registra los rechazos en una transacción propia (petición 18), y nadie puede modificarla sin que la verificación de huellas lo detecte (petición 19 y Paso 5 de la demo). AC-02: si el núcleo no responde, la transferencia queda FALLIDA con 503 y sin efecto parcial, se puede reintentar con el mismo código (PUT) y un código repetido nunca duplica la operación (409). AC-03: el núcleo se cambia con una variable (SITRA_NUCLEO_MODO) y las pruebas unitarias corren sin núcleo y sin base de datos.

Si el proyecto pasara a producción real, lo primero que evolucionaría es el conector real del núcleo, en cuanto el cliente confirme cómo expone sus datos, junto con la conciliación diaria (R-02 y R-06). Después, el endurecimiento del despliegue: HTTPS, el usuario sitra_app de mínimo privilegio, un secreto JWT definido y rotado, MariaDB 10.11 y respaldos automáticos (R-01, R-04, R-05 y R-07). Ninguno de esos cambios toca las capas de presentación ni de negocio: esa es la prueba de que la estructura elegida funciona.

Lista de chequeo de entrega

#	Entregable	¿Incluido?	Dónde
1	DAS completo (Secciones 1–6) en PDF dentro de docs/	Sí	docs/DAS.pdf y docs/DAS.docx (v1.0)
2	3 escenarios de calidad ISO 25010 con métricas	Sí	Sección 1.3 (AC-01, AC-02, AC-03)
3	5 vistas: Contexto, Contenedores, Clases, Secuencia, Despliegue	Sí	Sección 3 y docs/diagramas/
4	Mínimo 1 ADR en docs/adr/ con la plantilla oficial	Sí (5)	docs/adr/ADR-001 a ADR-005
5	Repositorio Git con ≥ 10 commits atómicos y .gitignore correcto	Sí	Historial con Conventional Commits; .gitignore excluye .env
6	Script SQL DDL con PK, FK, CHECK, UNIQUE y borrado lógico	Sí	db/migrations/V1 a V4 (Sección 5.2)
7	CRUD punta a punta ejecutable (Modelo–Controlador–Rutas–API)	Sí	Sección 5.3; src/main/java
8	Colección Postman o pruebas de integración pasando	Sí (ambas)	postman/ y src/test/java (Sección 5.4)
9	README.md que permite instalar y ejecutar sin ayuda del autor	Sí	README.md, sección "Instalar y ejecutar"
10	Análisis de trade-offs y ≥ 3 puntos de sensibilidad	Sí (5 y 7)	Secciones 6.1 y 6.2